

Zenject implementation overview

General summary

Zenject was integrated into the Unity project to move dependency creation and wiring out of gameplay scripts and into installers. The project now uses a global ProjectContext for application-wide services and scene-level SceneContext installers for objects that belong only to a specific scene.

This makes the code less dependent on direct singleton access and hard references, because scripts can receive the objects they need through [Inject].

ProjectContext

A ProjectContext prefab was added at:

`Assets/Resources/ProjectContext.prefab`

This prefab contains the global Zenject context and has ProjectInstaller attached as its installer. Because it is placed in Assets/Resources, Zenject can load it automatically when the application starts.

ProjectInstaller

The global installer was added at:

`Assets/Scripts/ProjectInstaller.cs`

It registers services that are useful across the whole project:

- 'ISceneLoader' -> 'UnitySceneLoader'
- 'ISaveSystem' -> 'PlayerPrefsSaveSystem'
- 'IAudioService' -> 'NullAudioService'

All of these are registered with AsSingle, because they represent shared project-level services and do not need multiple instances.

Global services

Scene loading

ISceneLoader abstracts Unity scene loading.

Implementation:

`Assets/Scripts/UnitySceneLoader.cs`

This replaces direct usage of `SceneManager.LoadSceneAsync` where practical. For example, `LoadingScreenController` now receives `ISceneLoader` through Zenject and uses it to load the target scene.

Save system

`ISaveSystem` was added as a simple abstraction over saving and loading values.

Implementation:

`Assets/Scripts/PlayerPrefsSaveSystem.cs`

It currently uses Unity `PlayerPrefs`, but because the rest of the project depends on `ISaveSystem`, the implementation can later be replaced without changing gameplay scripts.

Audio service

`IAudioService` defines the audio actions used by gameplay code.

There are two implementations:

- '`NullAudioService`', registered globally in '`ProjectContext`'
- '`GameAudioManager`', registered in the game scene

`NullAudioService` is a safe fallback for scenes where no real audio manager exists. In the actual game scene, `GameSceneInstaller` rebinds `IAudioService` to the real `GameAudioManager`.

Game scene installer

The scene installer is located at:

`Assets/Scripts/GameSceneInstaller.cs`

It is connected through `SceneContext` in `Assets/Scenes/Game.unity`.

This installer registers scene-specific dependencies:

- '`IBallRegistry`' -> '`BallRegistry`'
- scene transforms such as camera point and spawned balls parent
- scene components such as '`BallPlacer`', '`IslandManager`', '`CameraShake`', and '`LoseLineGameOver`'
- '`IAudioService`' -> '`GameAudioManager`' for the game scene

Scene objects stay in the scene installer because they depend on scene hierarchy objects and should not be global.

Loading scene

Assets/Scenes/Loading.unity now has a SceneContext, so Zenject can inject dependencies into LoadingScreenController.

LoadingScreenController uses [InjectOptional] for ISceneLoader. This allows it to work through Zenject when the container is available, while still keeping a fallback path for direct Unity scene loading.

Replaced direct dependencies

Several scripts were adjusted to receive dependencies from Zenject instead of manually searching for or storing global singleton objects.

Updated scripts include:

- 'BallPlacer'
- 'IslandManager'
- 'CameraShake'
- 'LoseLineGameOver'
- 'LoadingScreenController'
- 'GameAudioManager'

The old direct dependency on GameAudioManager.Instance was removed. Gameplay scripts now depend on IAudioService, which is cleaner and easier to replace or test.

Lifetime choices

AsSingle was used for global services because one shared instance is enough for the whole application:

- 'UnitySceneLoader' does not need state per caller.
- 'PlayerPrefsSaveSystem' represents one save access layer.
- 'NullAudioService' is only a fallback object.
- 'BallRegistry' is scene-scoped, but one registry per game scene is enough.

AsTransient was not needed because these services are not temporary objects that should be recreated every time they are requested.

AsCached was not necessary because the services are intentionally single shared instances, not lazily cached variants of multiple bindings.

Benefit for the project

The main benefit is that scripts no longer need to know how

their dependencies are created. They only declare what they need, and Zenject provides it.

This improves separation of responsibilities:

- gameplay scripts focus on gameplay behavior
- installers decide which implementation is used
- global services live in 'ProjectContext'
- scene-specific objects live in 'SceneContext'

It also makes the project easier to extend. For example, `PlayerPrefsSaveSystem` could later be replaced with a file-based save system, or `UnitySceneLoader` could be replaced with a loading service that supports transitions, without rewriting the scripts that use those services.